sas innovate

PROC SQL: Zero to Hero

Carleigh Jo Crabtree
Technical Training Consultant

#SASinnovate

Contents

Abstract

PROC SQL: Zero to Hero shows how to query and subset data, create and combine tables, and summarize results using PROC SQL. Join this session to see practical PROC SQL code that accomplishes common data tasks from simple queries to more advanced techniques.

Data Dictionaries

You will work with three datasets containing customer, item, and order information from the fictitious company Chocolate Enterprise.

CHOC_ENTERPRISE_CUSTOMER

Variables in Creation Order

#	Variable	Type	Len	Format	Informat	Label
1	customer_self_description	Char	64			Customer Self-Description
2	loyalty_program	Char	200			Card Program Membership
3	customer_rk	Num	8	12.	12.	Customer Key
4	customer_name	Char	52	$CHAR52.	$CHAR26.	
5	Email	Char	82	$CHAR82.	$CHAR41.	
6	bday_month	Num	8			
7	age	Num	8			

CHOC_ENTERPRISE_ITEM

Variables in Creation Order

#	Variable	Type	Len	Format	Informat	Label
1	item_rk	Num	8	12.	12.	Item Key
2	product_line	Char	100			Item Product Line
3	category	Char	100			Item Category
4	package	Char	32			Item Package Type
5	item_desc	Char	100			Item Description

CHOC_ENTERPRISE_ORDERS

Variables in Creation Order

#	Variable	Type	Len	Format	Informat	Label
1	continent	Char	40			Continent
2	transaction_id	Char	32			Transaction ID
3	date	Num	8	DATE8.		Date
4	date_year	Num	8	YEAR.		Date (Year)
5	total_line_item_cost	Num	8	DOLLAR12.4	NLNUM18.5	Total Line Item Cost
6	transaction_line_item_no	Num	8			Transaction Line Item No
7	retail_transaction_type_cd	Char	3			Retail Transaction Type Code
8	order_channel	Char	32			
9	order_type	Char	8			
10	total_line_item_sale_amt	Num	8	DOLLAR12.2	NLNUM18.5	Total Line Item Sale Amount
11	item_qty	Num	8			Item Quantity
12	list_price_amt	Num	8	DOLLAR12.4	NLNUM18.5	List Price Amount
13	item_cost_amt	Num	8	DOLLAR12.4	NLNUM18.5	Item Price Amount
14	item_rk	Num	8	12.	12.	Item Key
15	store_id	Num	8			
16	customer_rk	Num	8	12.	12.	Customer Key
17	country_cd	Char	3			Country Code
18	country_nm	Char	50			Country Name

Variables in Creation Order						
#	**Variable**	**Type**	**Len**	**Format**	**Informat**	**Label**
19	state_region_cd	Char	3			State Region Code
20	state_region_nm	Char	50			
21	county_nm	Char	100			County Name
22	city_nm	Char	100			City Name
23	postal_cd	Char	30			Postal Code
24	retail_outlet_name	Char	40			Retail Outlet Name
25	retail_outlet_format_cd	Char	32			Retail Outlet Format Code
26	retail_outlet_type_cd	Char	32			Retail Outlet Type Code
27	store_size	Num	8	NLNUM10.2	NLNUM10.2	Total Size
28	address_line_1	Char	100			Address Line 1 Text

Exploring SAS Studio

1. Launch Google Chrome from the desktop and select **SAS Studio** from the Bookmarks bar.
2. Enter **student** in the **User ID** field and **Metadata0** in the **Password** field. Click **Sign in**. Select **Yes** when prompted to opt in to all assumable groups.
3. Click the **applications menu** icon  in the upper left corner. The applications menu shows all the available applications. Depending on your environment, these applications might differ. Select **Develop Code and Flows** to open SAS Studio. We use SAS Studio to write and submit code in this workshop.
4. The Start Page tab is open by default, and it provides convenient links to start writing a program or use other point-and-click tools, including importing and querying data. Click **Program in SAS** to open a new tab with the Program Editor. Each item that you open in SAS Studio has its own tab.
5. The navigation pane is on the left side of SAS Studio. Each section in the navigation pane provides access to different resources in your SAS environment.
    - **Open Items**  enables you to quickly view and access files that are currently open in your SAS Studio session. You can see that the list of open files matches the tabs.
    - **SAS Server**  provides access to files and folders in your SAS environment. Keep in mind that the files and folders that you see here are not on your local machine. They're on the server where SAS is running. If you have the available permissions, you can use the **Upload files**  button to load local files to the SAS server. The SAS Server also enables you to perform other tasks, such as background submit and schedule programs and basic file operations, such as copy, paste, and delete.
    - **Steps**  enables you to build flows in SAS Studio. In a flow, you can build a sequence of steps, including programs, queries, and data transformations, as well as your own custom steps.
    - **Tasks**  enables you to access tasks in SAS Studio. Tasks are based on SAS procedures and generate SAS code and formatted results for you.
    - **Snippets**  enables you to access snippets. Snippets are lines of commonly used code or text that you can save and reuse. SAS Studio provides several predefined snippets, and you can create your own.
    - **Library Connections**  provides access to your SAS libraries. You can open SAS tables and add them to your programs. You can even expand a table and view its columns.
    - **Git Repositories**  enables you to access the basic Git features from within SAS Studio for maintaining and viewing a history of program edits.

Demo 1: Examine Statements

6. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D01_ExamineStatements.sas**.
7. Demo 1 explores PROC SQL clauses and frequently used options and functions.
8. Make the following modifications to the program:

```
/* Print the first 100 rows of sqw.choc_enterprise_customer to explore the data.*/
/* SELECT *: Selects all columns from the input table. */
/* INOBS: Limits the number of observations read in for processing. */
proc sql inobs=100;
select *
    from sqw.choc_enterprise_customer;
quit;

/* What are the column names and attributes? */
/* DESCRIBE TABLE: Prints column names and attributes to the log. */
proc sql;
describe table sqw.choc_enterprise_customer;
quit;

/* What are the distinct values in the loyalty_program column? */
/* SELECT DISTINCT: Selects unique values in the column(s). */
proc sql;
select distinct
    from sqw.choc_enterprise_customer;
quit;
```

```
/* Generate a list of customers that are not currently Chocolate Club Members. */
/* WHERE clause: Filters data.                                                  */
proc sql;
select customer_name, email
    from sqw.choc_enterprise_customer
    where loyalty_program="Non-Member" and email is not null;
quit;


/* How many total customers are not currently Chocolate Club Members? */
/* COUNT(*): Returns a count of all rows in the table including null values. */
/* Create new columns on the SELECT clause and assign them a name after the AS
keyword. */
/* Assign column attributes to columns on the SELECT clause. */
proc sql;
select count(*) as TotalNonMembers format=comma16.
    from sqw.choc_enterprise_customer
    where loyalty_program="Non-Member";
quit;


/* How many Non-Members have birthdays each month?                              */
/* GROUP BY clause: Enables processing data in groups.                          */
/* COUNT(colName): Returns the number of rows that do not have a null value.    */
/* COUNT(colName) with GROUP BY: Returns the number of rows within each group. */
proc sql;
select bday_month label="Birthday Month", count(customer_name) as TotalBdays
format=comma10.
    from sqw.choc_enterprise_customer
    where loyalty_program="Non-Member"
    group by bday_month;
quit;
```

```
/* Filter the report for birthday months with more than 6,000 customers to market
to. */

/* HAVING clause: Works with GROUP BY to filter grouped data. */

proc sql;

select bday_month label="Birthday Month", count(customer_name) as TotalBdays
format=comma10.

    from sqw.choc_enterprise_customer

    where loyalty_program="Non-Member"

    group by bday_month

    having TotalBdays> 6000;

quit;


/* Which month has the highest number of birthdays? */

/* ORDER BY clause: Determines the order of rows.   */

proc sql;

select bday_month label="Birthday Month", count(customer_name) as TotalBdays
format=comma10.

    from sqw.choc_enterprise_customer

    where loyalty_program="Non-Member"

    group by bday_month

    having TotalBdays> 6000

    order by TotalBdays desc;

quit;
```

Demo 2: Case Expression

9. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D02_CaseExpresison.sas**.
10. Demo 2 explores manipulating data with a simple CASE expression.
11. Make the following modifications to the program:

```
/* Clean the customer_name column so it only contains the customers name without
prefixes or commas. */


/* CREATE TABLE: Creates a table rather than a report. */

/* FIND function: Returns the position at which the specified string begins. */

/* When the value returned from the FIND function is greated than 0, the value was
found in the string. */

proc sql;

create table out.customers as

select customer_name,

       find(customer_name, 'Mrs.', 'i') as Mrs,

       find(customer_name, 'Mr.', 'i') as Mr,

       find(customer_name, 'Ms.', 'i') as Ms,

       find(customer_name, 'Miss', 'i') as Miss,

       find(customer_name, ',') as Comma,

       email, bday_month, age, customer_self_description, loyalty_program,
customer_rk

    from sqw.choc_enterprise_customer;

quit;
```

```
/* Simple CASE Expression: Used to assign values to a new column conditionally. */

/* TRANWRD function: Replaces substring with designated string. */

/* STRIP function: Removes leading and trailing blanks. */

/* SCAN function: Extracts the specified word within a string. */

/* CATX function: Concatenates specified strings and inserts delimiter between
them. */

proc sql;

create table out.customers_cleaned as

select customer_name,

    case

        when Mrs>0 then strip(tranwrd(customer_name, 'Mrs.', ' '))

        when Mr>0 then strip(tranwrd(customer_name, 'Mr.', ' '))

        when Ms>0 then strip(tranwrd(customer_name, 'Ms.', ' '))

        when Miss>0 then strip(tranwrd(customer_name, 'Miss', ' '))

        when Comma>0 then catx(' ', scan(customer_name, 2, ','),
scan(customer_name, 1, ','))

        else customer_name

    end as Customer_Names_Cleaned label="Customer Name",

    email, bday_month, age, customer_self_description, loyalty_program,
customer_rk

    from out.customers;

quit;
```

Demo 3: Dictionary Tables

12. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D03_DictionaryTables.sas**.
13. Demo 3 explores dictionary tables which contain session metadata.
14. Make the following modifications to the program:

```
/* View the column names and labels in dictionary.columns. */

/* Dictionary tables: Contain session metadata about libraries, tables, columns
etc. */

/* DICTIONARY.columns: Provides detailed information about all columns in all
tables. */

proc sql;

describe table dictionary.columns;

quit;


/* What columns could be primary/ foreign keys for joining tables? */

proc sql;

select *

    from dictionary.columns

    where libname="SQW" and memname like "CHOC%";

quit;


/* Which tables have the item_rk column? */

proc sql;

select libname, memname, name, type, length, label

    from dictionary.columns

    where libname="SQW" and memname like "CHOC%" and name="item_rk";

quit;
```

```
/* Which tables have the customer_rk column? */
proc sql;
select libname, memname, name, type, length, label
    from dictionary.columns
    where name="customer_rk";
quit;
```

Demo 4: Inner Join

15. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D04_InnerJoin.sas**.
16. Demo 4 explores joining three datasets with an inner join.
17. Make the following modifications to the program:

```
/* Join sqw.choc_enterprise_orders, sqw.choc_enterprise_items and
out.customers_cleaned tables. */


/* Put columns names into macro variables to use on SELECT clause. */
proc sql;
select name
     into :ord_col separated by ","
     from dictionary.columns
     where libname="SQW" and memname="CHOC_ENTERPRISE_ORDERS";
quit;


proc sql;
select name
     into :item_col separated by ","
     from dictionary.columns
     where libname="SQW" and memname="CHOC_ENTERPRISE_ITEM";
quit;


%put &item_col;
```

```
proc sql;
select name
     into :cust_col separated by ","
     from dictionary.columns
     where libname="OUT" and memname="CUSTOMERS_CLEANED";
quit;


%put &ord_col , &item_col , &cust_col;


/* INNER JOIN: Returns matching rows based on join
criteria.                                        */
/* Join all 3 tables at the same time to create out.ordItemsCust. */
proc sql;
create table out.ordItemsCust as
select
continent,transaction_id,date,date_year,total_line_item_cost,transaction_line_item
_no,retail_transaction_type_cd,order_channel,
        order_type,total_line_item_sale_amt,item_qty,list_price_amt,item_cost_amt,
o.item_rk,store_id,o.customer_rk,country_cd,country_nm,state_region_cd,
        state_region_nm,county_nm,city_nm,postal_cd,retail_outlet_name,retail_outl
et_format_cd,retail_outlet_type_cd,store_size,address_line_1,
        product_line,category,package,item_desc,customer_name,Customer_Names_Clean
ed,Email,bday_month,age,customer_self_description,
        loyalty_program
    from sqw.choc_enterprise_orders as o
        inner join sqw.choc_enterprise_item as i
    on o.item_rk = i.item_rk
        inner join out.customers_cleaned as c
    on o.customer_rk=c.customer_rk;
quit;
```

Demo 5: Non-correlated Subqueries

18. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D05_NoncorrelatedSubqueries.sas**.
19. Demo 5 explores how to build non-correlated subqueries.
20. Make the following modifications to the program:

```
/* Generate a table of the customers that are Non-Members that have ordered
greater than the average number of items ordered. */


/* Subquery in HAVING clause: Returns values to be used in the outer query's
HAVING clause. */

/* Must return a value or values from only one column. */

/* Noncorrelated subquery: Executes independently of the outer query. */


/* Step 1: Calculate the average items ordered for Non-Members older than 21. */

proc sql;

select avg(item_qty) as AvgItemsOrdered

    from out.ordItemsCust

    where loyalty_program="Non-Member" and age>21;

quit;


/* Step 2: Calculate the total number of items each customer has ordered. */

/* Filter the report for customers that have ordered more than the average number
of items ordered. */

proc sql;

select distinct customer_names_cleaned, email, sum(item_qty) as TotalItemsOrdered

    from out.ordItemsCust

    where loyalty_program="Non-Member" and age>21 and email is not null

    group by customer_name, email

    having TotalItemsOrdered> 12.58922

    order by totalitemsordered desc;

quit;
```

```
/* Step 3: Combine the query and subquery. */
proc sql;
select distinct customer_names_cleaned, email, sum(item_qty) as TotalItemsOrdered
    from out.ordItemsCust
    where loyalty_program="Non-Member" and age>21 and email is not null
    group by customer_name, email
    having TotalItemsOrdered> (select avg(item_qty) as AvgItemsOrdered
                                    from out.ordItemsCust
                                    where loyalty_program="Non-Member" and age>21)
    order by totalitemsordered desc;
quit;


/* What happens when new data is added to the table that changes the average
generated by the subquery? */


/* Add rows to out.ordersItemsCustomers to change the AverageItemsOrdered value.*/
/* INSERT INTO: Adds rows to an existing table. */
/* VALUES clause: Assign values to columns by position. */
proc sql;
insert into out.ordersItemsCustomers
    (Customer_Names_Cleaned, email, loyalty_program, age, item_qty, item_desc)
    values ("Carleigh Jo Crabtree", "CarleighJo.Crabtree@sas.com", "Non-Member",
55, 5000, "Raspberry Chocolate Permiere")
    values ("Carleigh Jo Crabtree", "CarleighJo.Crabtree@sas.com", "Non-Member",
55, 10000, "Dark Chocolate Raspberry Starfish");
/*  values ("Carleigh Jo Crabtree", "CarleighJo.Crabtree@sas.com", "Non-Member",
55, 25000, "4 pc White Wedding / Party Favor with White Ribbon"); */
quit;
```

```
/* View new average. */
proc sql;
select avg(item_qty) as AvgItemsOrdered
    from out.ordersItemsCustomers
    where loyalty_program="Non-Member" and age>21;
quit;


/* Run the query. */
/* Uncomment the third values statement to change the average again. Re-run the
query. */
/* Notice the data is updated because the subquery accesses the current data
rather than a hard coded value. */
proc sql;
select distinct customer_names_cleaned, email, sum(item_qty) as TotalItemsOrdered
    from out.ordersItemsCustomers
    where loyalty_program="Non-Member" and age>21 and email is not null
    group by customer_name, email
    having TotalItemsOrdered> (select avg(item_qty) as AvgItemsOrdered
                                    from out.ordersItemsCustomers
                                    where loyalty_program="Non-Member" and age>21)
    order by totalitemsordered desc;
quit;
```

Demo 6: Macro Variables

21. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D06_MacroVariables.sas**.
22. Demo 6 explores how to create data driven macro variables with PROC SQL.
23. Make the following modifications to the program:

```
/* What is the average amount a customer will spend on one item per order? Put the
value into a macro variable. */

/* INTO clause: Stores values as macro variables. */

/* Notice extra spaces in the macro. By default, macro variables with numeric
values are formatted with the BEST8. format. */

/* TRIMMED: Used to trim leading and trailing blanks. */

proc sql;

select round(avg(total_line_item_sale_amt)) as AvgSpent

    into :AvgSpent

/*  into :AvgSpent trimmed */

    from out.orditemscust;

quit;

%PUT The average amount a customer spends on one item per order is &AvgSpent;


/* Use the macro variable to count how many customers spent more than the average
on sugar free items. */

proc sql ;

title "Total Number of Sugar Free Orders Where the Sale Amount was Greater than
Average";

select count(customer_names_cleaned) as SugarFreeOrders format=comma16.

    from out.orditemscust

    where category="Sugar Free" and total_line_item_sale_amt> &AvgSpent;

title;

quit;
```

```
/* Create a table with the total profit for each category within each product
line. */

proc sql;

create table out.ProductProfit as

select product_line, category, sum(total_line_item_sale_amt) as CategoryProfit
format=dollar25.

    from out.orditemscust

    group by product_line, category;

quit;


/* Store the distinct values of product_line in a series of macro variables. */

proc sql;

select distinct product_line

    into :Product1-

    from out.orditemscust;

quit;

%PUT _USER_;
```

```
/* Use the table and macro variables created to create bar charts of the profit by
category for each product line. */

%macro CategoryProfits;


%local i;

%do i=1 %to 6;

title "Profit by Category for the &&product&i Product Line";

proc sgplot data=out.productprofit noautolegend ;

    hbar category / response=categoryprofit stat=sum group=category
categoryorder=respdesc;

    where product_line="&&product&i";

    format categoryprofit dollar8.;

run;

title;

%end;


%mend;


/* Call the macro program to view the bar charts. */

ods graphics on;

%CategoryProfits


/* Create a macro variable list of the categories in the Assorted product line
where profit is greater than $500,000. */

/* Include quotation marks around the values so the list can be used for
filtering. */

proc sql;

select quote(strip(category))

    into :TopAssortedCategories separated by ","

    from out.productprofit

    where product_line="Assorted" and categoryprofit> 500000;

quit;
```

```
%PUT &=TopAssortedCategories;


/* Create a table of the items that make up the top sales in the assorted product
line. */

proc sql;

create table out.TopAssorted as

select distinct product_line, category, package, item_desc

    from out.orditemscust
```

Demo 7: Summary Functions

24. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D07_SummaryFunctions.sas**.
25. Demo 7 explores summarizing down a column and across rows of data.
26. Make the following modifications to the program:

```
/* Create out.stateProductLines to use for summary functions. */

/* Calculates how many orders were placed in each state for each product line. */

proc sql;

create table out.StateProductLines as

select distinct state_region_nm, product_line, count(product_line) as
productlinetotal format= comma20.

    from out.orditemscust

    where state_region_nm is not null

    group by state_region_nm, product_line

    order by state_region_nm, productlinetotal desc;

quit;


/* Transpose out.stateProductLines so product lines are columns. */

proc transpose data=out.stateproductlines out=out.spl_transposed(rename=("Misc
Pack"n=Misc_Pack) drop=_name_);

    var productlinetotal;

    id product_line;

    by state_region_nm;

run;
```

```
/* COALESCE function: Returns the first non-null value from a list of arguments.*/
proc sql;
update out.spl_transposed
    set
        Snacks= coalesce(Snacks, 0);
quit;


/* Summarize down a column: */
/* Calculate the maximum, minimum and mean number of orders placed for the
Assorted product line. */
proc sql;
select max(Assorted) as HighestAssortedOrders format= comma16.,
       min(Assorted) as LowestAssortedOrders format= comma16.,
       mean(Assorted) as AverageAssortedOrders format= comma16.
    from out.spl_transposed;
quit;


/* Summarize across rows: */
/* Calculate the maximum, minimum and mean number of orders placed across all
product lines for each state. */


/* When summarizing across rows, column lists are not valid. */
proc sql;
select state_region_nm,
       max(of Assorted-- Snacks) as HighestOrders format= comma16.,
       min(of Assorted-- Snacks) as LowestOrders format= comma16.,
       mean(of Assorted-- Snacks) as AverageOrders format= comma16.
    from out.spl_transposed;
quit;
```

```
/* Use data driven macros and dictionary.columns to put column names in a macro
variable list separated by commas. */
proc sql;
select name
     into :colNames separated by ","
     from dictionary.columns
     where libname="OUT" and memname="SPL_TRANSPOSED";
quit;

%put &colNames;
%let pl= Assorted,Misc_Pack,Candy,Drinks,Books,Snacks;
%put &=pl;

/* Use the macro variable, rather than typing column names. */
proc sql;
select state_region_nm label= "State Name",
       max(&pl) as HighestOrders format= comma16.,
       min(&pl) as LowestOrders format= comma16.,
       mean(&pl) as AverageOrders format= comma16.
    from out.spl_transposed;
quit;
```

Demo 8: Boolean Expressions

27. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D08_BooleanExpressions.sas**.
28. Demo 8 explores boolean expressions.
29. Make the following modifications to the program:

```
/* How many customers in each state are Chocolate Club Members? How many are Non-
Members? */


/* Step 1: Use find function to determine if Chocolate Club Member is found in the
loyalty_program column. */

/* If it is found, the value will be greater than 0. If it is not found, the value
will be 0. */

proc sql inobs=100;

select state_region_nm label= "State Name", loyalty_program,
customer_names_cleaned,

    find(loyalty_program, "Chocolate Club Member", "i")

    from out.orditemscust

    where state_region_nm is not null;

quit;


/* Step 2: Use boolean expression. */

/* Boolean expression: Evaluate to true (1) or false (0).*/

proc sql inobs=100;

select state_region_nm label= "State Name", customer_names_cleaned,

        find(loyalty_program, "Chocolate Club Member", "i")>0 as
ChocolateClubMember format= comma16.,

        find(loyalty_program, "Chocolate Club Member", "i")=0 as NonMember format=
comma16.

    from out.orditemscust

    where state_region_nm is not null;

quit;
```

```
/* Step 3: Use boolean expression with SUM function to calculate how many members
or non-members are in each state. */

proc sql;

select state_region_nm label= "State Name",

        sum(find(loyalty_program, "Chocolate Club Member", "i")>0) as
ChocolateClubMembers format= comma16.,

        sum(find(loyalty_program, "Chocolate Club Member", "i")=0) as NonMembers
format= comma16.

    from out.orditemscust

    where state_region_nm is not null

    group by state_region_nm

    order by nonmembers desc;

quit;
```

Demo 9: SET Operators

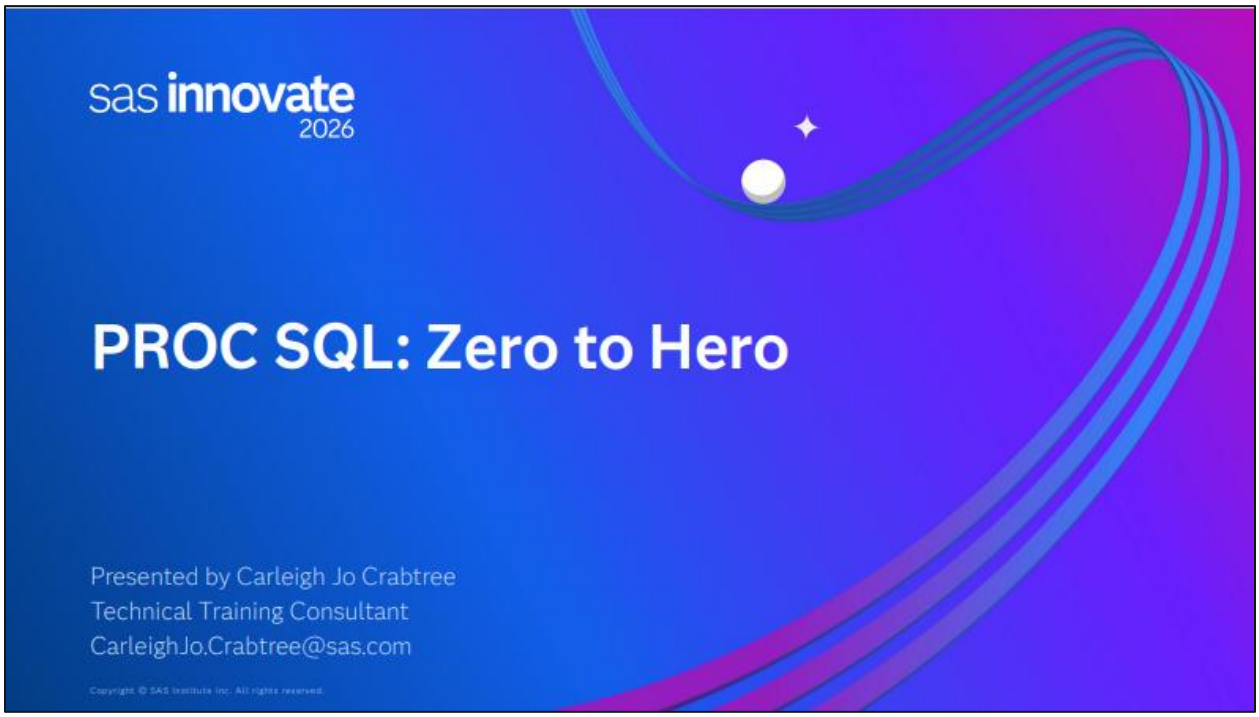
30. In SAS Server, expand **Files** > **Home** > **workshop** > **SQL_Prog**. Double-click **D09_SetOperators.sas**.
31. Demo 9 explores SET operators as an alternative to joins and subqueries.
32. Make the following modifications to the program:

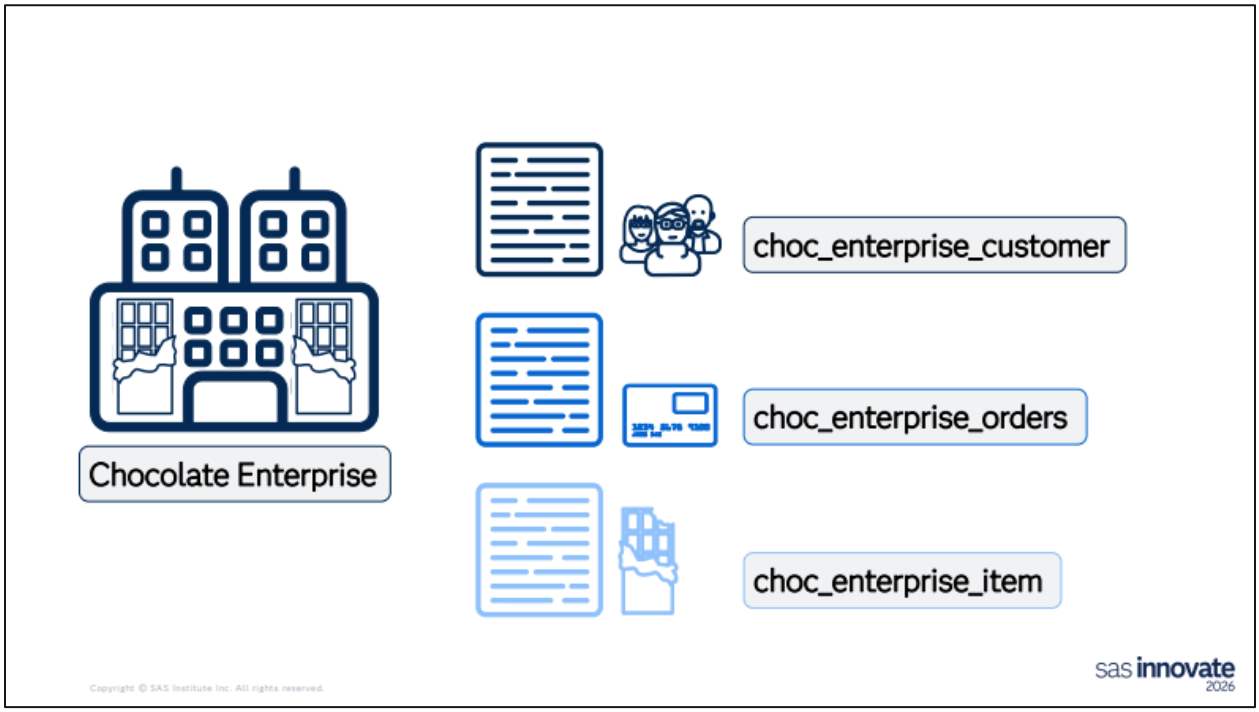
```
/* Create two tables. One for customers who ordered chocolate bars, one for
customers that ordered jelly beans. */

proc sql;

create table out.ChocolateBars as

select *

    from out.ordItemsCust

    where category="Chocolate Bars";

create table out.JellyBeans as

select *

    from out.orditemscust

    where category="Jelly Bean";

quit;


/* Create a list of customers that ordered both chocolate bars and jelly beans. */

/* INTERSECT operator: Result from both queries generated. */

/* Duplicate rows are removed from both intermediate result sets. */

/* Rows that are in the intermediate result sets are selected.    */

/* NUMBER: Includes the Row number column in the results. */
```

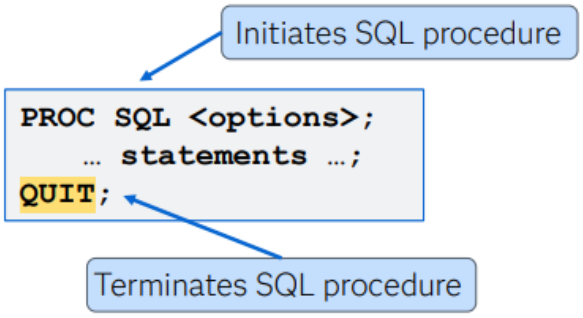
```
proc sql number;
title "Chocolate Bar and Jelly Bean Lovers";
select customer_names_cleaned
    from out.ChocolateBars
intersect
select customer_names_cleaned
    from out.JellyBeans;
title;
quit;

/* This can also be done with an inner join. */
proc sql number;
select distinct cb.customer_names_cleaned
    from out.chocolatebars as cb
        inner join out.jellybeans as jb
    on cb.customer_names_cleaned=jb.customer_names_cleaned;
quit;

/* Create a list of all customers that ordered chocolate bars, except those that
also ordered jelly beans. */
/* EXCEPT operator: Result from both queries generated. */
/* Duplicate rows are removed from both intermediate result sets. */
/* Rows from the first intermediate result set that are NOT in the second
intermediate result set are selected. */
```

```
proc sql number;
title "Chocolate Bar Lovers";
select customer_names_cleaned
    from out.ChocolateBars
except
select customer_names_cleaned
    from out.JellyBeans;
title;
quit;

/* This can also be done with a subquery. */
proc sql number;
select distinct customer_names_cleaned
    from out.chocolatebars
    where customer_names_cleaned not in (select customer_names_cleaned
                                         from out.jellybeans);
quit;
```

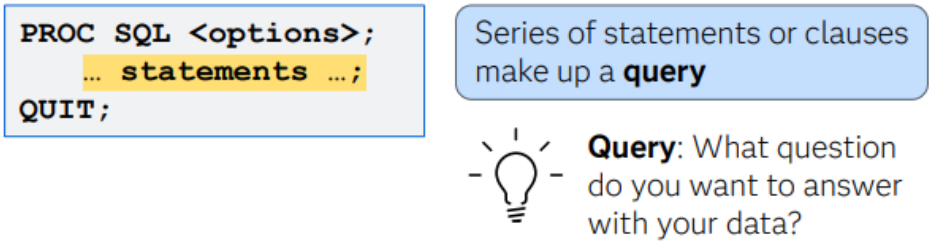
HOW Slides



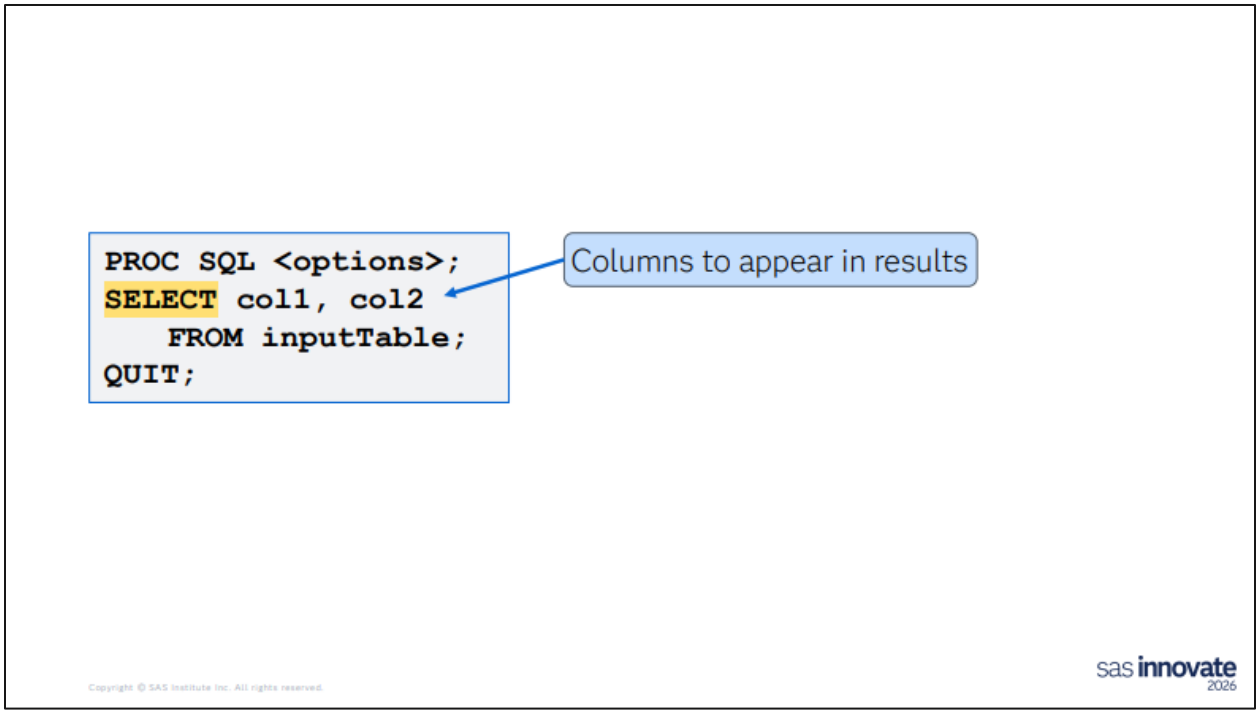




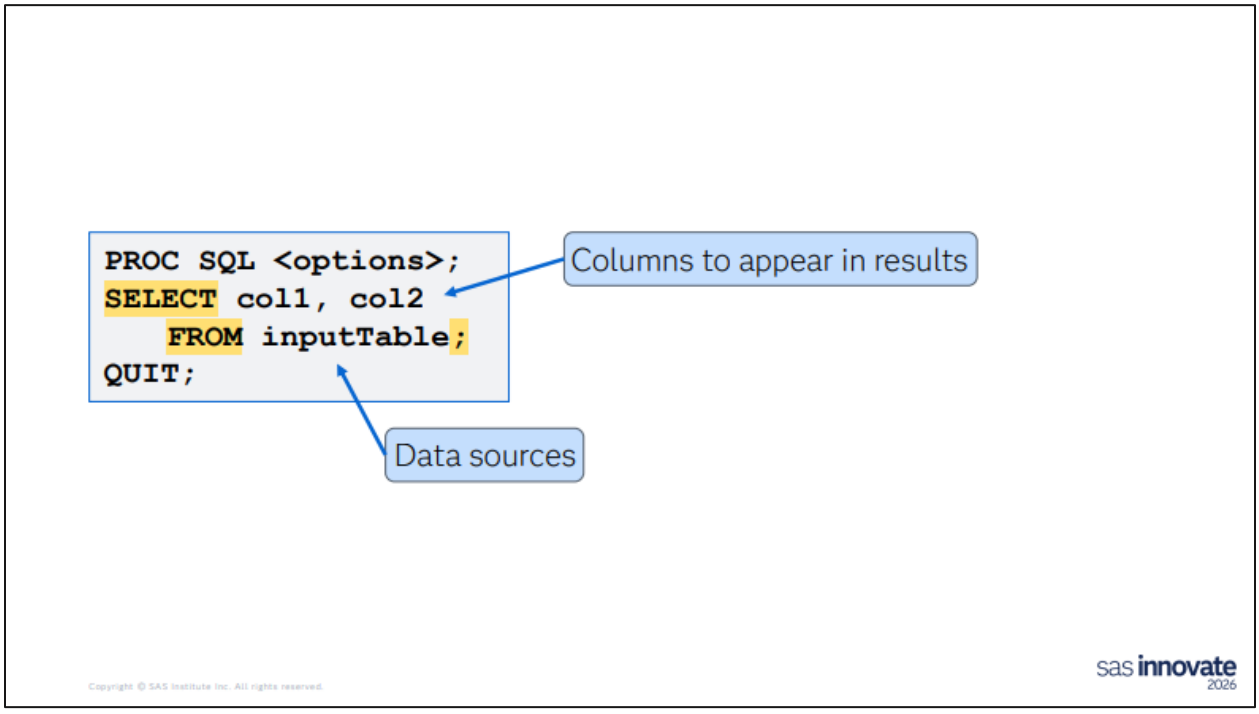
sas innovate 2026



sas innovate 2026



The SELECT clause selects columns to keep in the results.



The FROM clause selects one or more source tables or views.

Additional optional clauses:

```
PROC SQL <options>;
SELECT col1, col2
    FROM inputTable
    <WHERE clause>
    <GROUP BY clause>
    <HAVING clause>
    <ORDER BY clause>;
QUIT;
```

Mnemonic:
So
Few
Workers
Go
Home
On Boats

sas innovate 2026

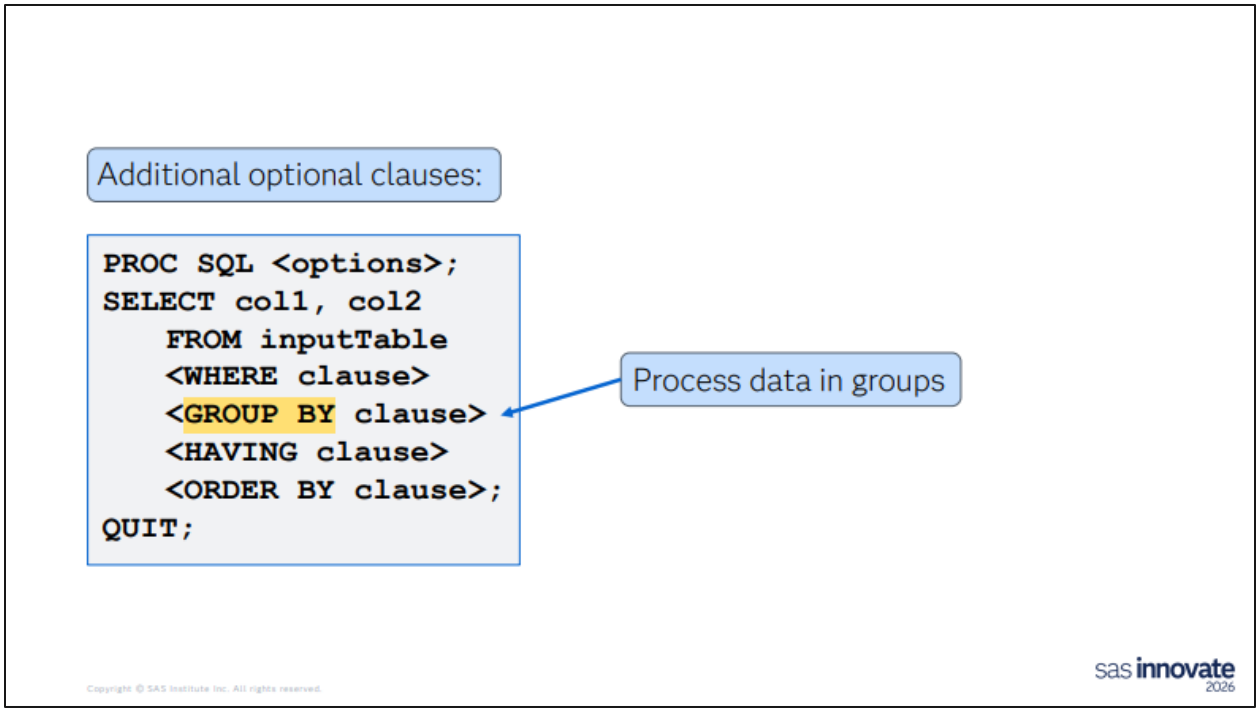
Additional optional clauses:

```
PROC SQL <options>;
SELECT col1, col2
    FROM inputTable
    <WHERE clause>
    <GROUP BY clause>
    <HAVING clause>
    <ORDER BY clause>;
QUIT;
```

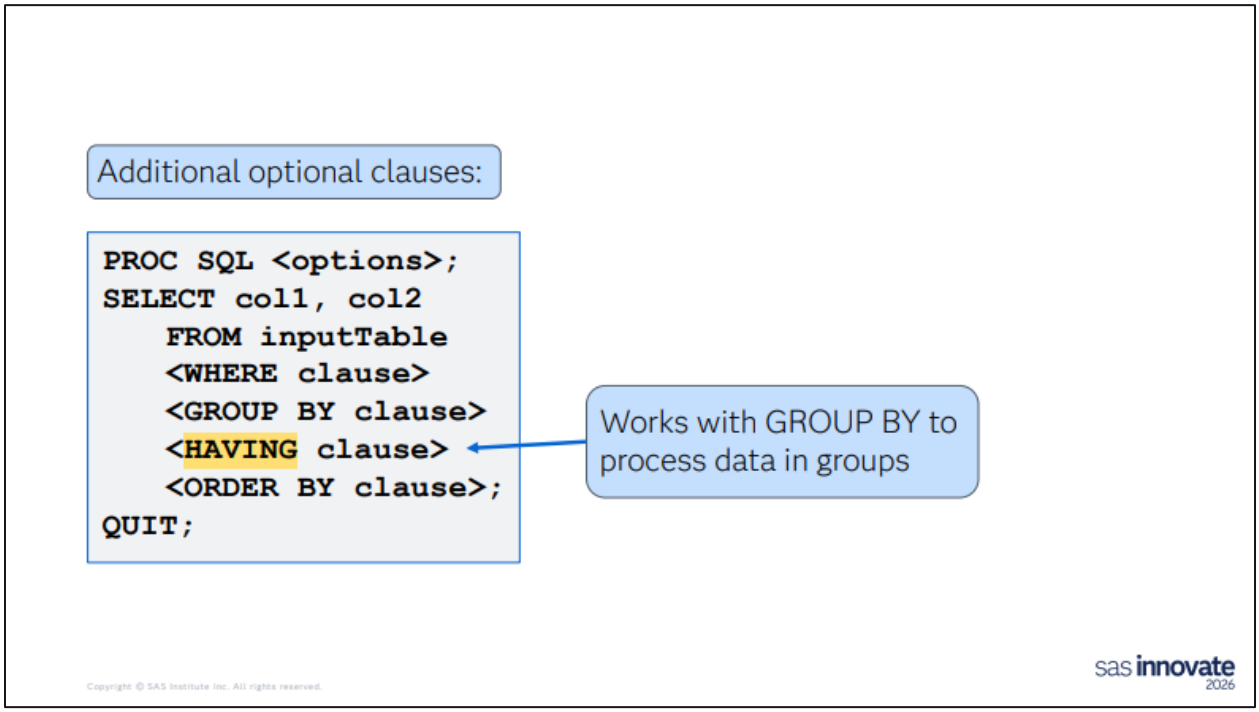
Filter rows of data

sas innovate 2026

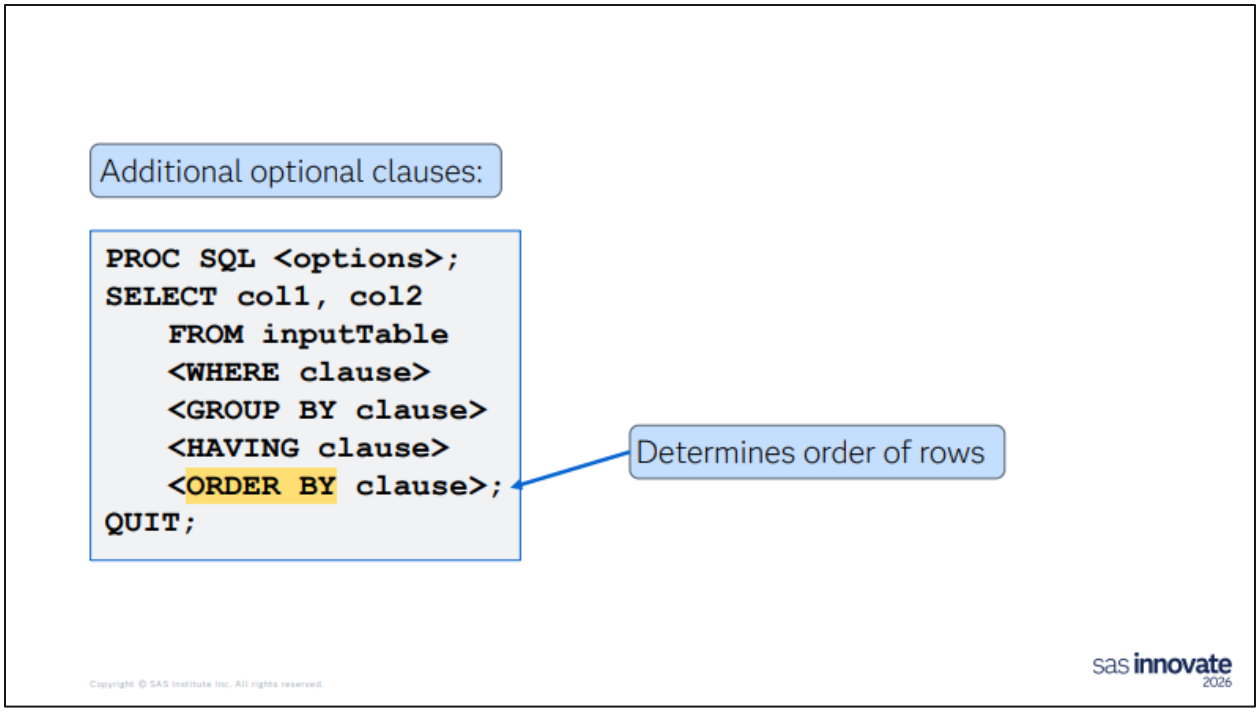
The WHERE clause enables you to filter rows of data.



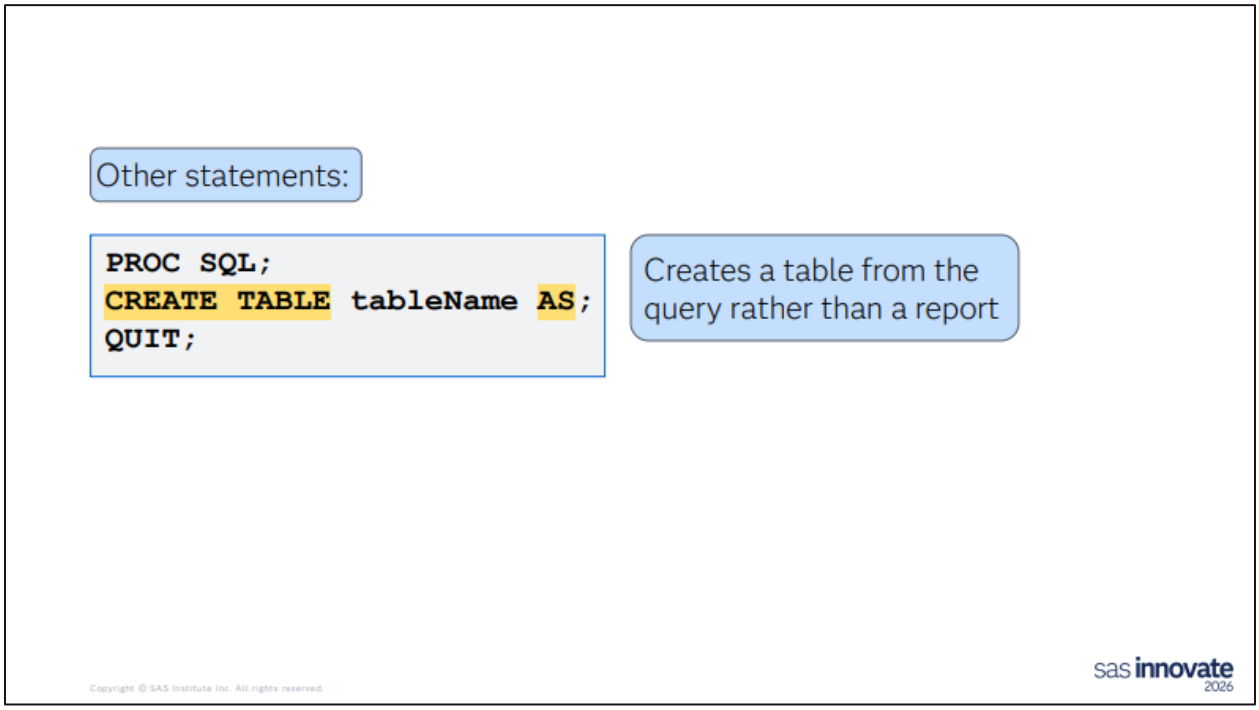
The GROUP BY clause enables you to process data in groups.



The HAVING clause works with the GROUP BY clause to filter grouped results.



The ORDER BY clause specifies the order of rows.



Other statements:

```
PROC SQL;
CREATE TABLE tableName AS;
QUIT;
```

Creates a table from the query rather than a report

```
PROC SQL;
DESCRIBE TABLE tableName;
QUIT;
```

Prints column names and attributes to the log

sas innovate 2026

Thank you!

Let's connect!

CarleighJo.Crabtree@sas.com

sas innovate 2026